

Trabajo Práctico N°1

Python: Filtros y comunicación serie

Fundación Fulgor

1. Introducción

El presente trabajo práctico tiene como objetivo implementar en Python un sistema para generar y analizar filtros digitales, incorporando además el uso de comunicación serie. Para poder probar el sistema sin hardware físico, se utilizó un puerto serie virtual mediante `loop://`. De esta manera, el programa puede enviar comandos, recibirlos como eco del puerto virtual y ejecutar acciones sobre el filtro configurado.

El desarrollo se organizó en tres partes principales: el módulo de generación de filtros, el módulo de comunicación serie y una interfaz gráfica para visualizar los resultados. La interfaz permite modificar parámetros del filtro, generar respuestas temporales y frecuenciales, comparar distintos filtros y visualizar una señal BPSK filtrada.

2. Objetivos

Los objetivos principales del trabajo son:

- Generar filtros Raised Cosine (RC) y Root Raised Cosine (RRC).
- Configurar los atributos del filtro mediante comandos enviados por puerto serie.
- Implementar una comunicación serie virtual para probar el sistema sin hardware externo.
- Graficar la respuesta temporal discreta del filtro.
- Graficar la respuesta en frecuencia del filtro.
- Comparar varios filtros en un mismo ploteo.
- Visualizar una señal BPSK filtrada.
- Agrupar los resultados en una interfaz gráfica clara y ordenada.

3. Estructura del proyecto

El proyecto se organizó en los siguientes archivos:

Archivo	Función dentro del proyecto
<code>filtro.py</code>	Contiene la clase encargada de generar los filtros RC y RRC, calcular coeficientes, respuesta temporal, respuesta en frecuencia y señal BPSK filtrada.
<code>com_serie.py</code>	Implementa la comunicación serie. En este caso se utiliza el puerto virtual <code>loop://</code> , que permite simular transmisión y recepción sin hardware real.
<code>main.py</code>	Versión por consola del programa. Permite enviar comandos por serie y observar las respuestas desde la terminal.
<code>app_gui.py</code>	Interfaz gráfica que integra todo el funcionamiento: configuración, comunicación serie, gráficos y registro de comandos.

Cuadro 1: Archivos principales del proyecto.

4. Descripción breve del código

4.1. Módulo `filtro.py`

El archivo `filtro.py` contiene la clase principal `RaisedCosineFilter`. Esta clase permite construir un filtro a partir de cuatro parámetros:

- **alpha**: factor de roll-off del filtro.
- **span**: duración del filtro medida en símbolos.
- **sps**: cantidad de muestras por símbolo.
- **rrc**: variable booleana que define si el filtro es Root Raised Cosine o Raised Cosine clásico.

La clase genera los coeficientes del filtro y permite obtener tanto la respuesta temporal como la respuesta en frecuencia. Además, incluye una función para generar una señal BPSK aleatoria y observar el efecto del filtrado.

4.2. Módulo `com_serie.py`

El archivo `com_serie.py` contiene una clase de comunicación serie. En lugar de utilizar un puerto físico, se emplea:

`loop://`

Este puerto virtual funciona como un eco: todo comando enviado vuelve como dato recibido. Esto permite simular el comportamiento de una comunicación serie real sin necesidad de conectar una FPGA, ESP32 u otro dispositivo externo.

4.3. Módulo `app_gui.py`

El archivo `app_gui.py` implementa la interfaz gráfica. Desde esta ventana se puede:

- Conectar el puerto serie virtual.
- Modificar `alpha`, `span`, `sps` y el tipo de filtro.
- Enviar comandos manualmente.
- Visualizar la respuesta temporal discreta.
- Visualizar la respuesta en frecuencia.
- Comparar filtros con distintos valores de `alpha`.
- Graficar una señal BPSK filtrada.
- Observar el registro de transmisión y recepción serie.

5. Comandos implementados

El sistema acepta comandos de texto enviados por puerto serie. Algunos ejemplos son:

Comando	Descripción
<code>status</code>	Muestra la configuración actual del filtro.
<code>set alpha 0.5</code>	Modifica el factor de roll-off del filtro.
<code>set span 10</code>	Modifica la duración del filtro en símbolos.
<code>set sps 10</code>	Modifica las muestras por símbolo.
<code>set type rrc</code>	Selecciona filtro Root Raised Cosine.
<code>set type rc</code>	Selecciona filtro Raised Cosine clásico.
<code>gen</code>	Genera nuevamente el filtro con los parámetros actuales.
<code>coef</code>	Muestra los coeficientes del filtro.
<code>plot both</code>	Actualiza los gráficos temporal y frecuencial.
<code>compare</code>	Compara varios filtros con distintos valores de <code>alpha</code> .
<code>signal</code>	Grafica una señal BPSK filtrada.
<code>export</code>	Exporta los coeficientes a un archivo de texto.

Cuadro 2: Comandos principales del sistema.

6. Diagrama de flujo del programa

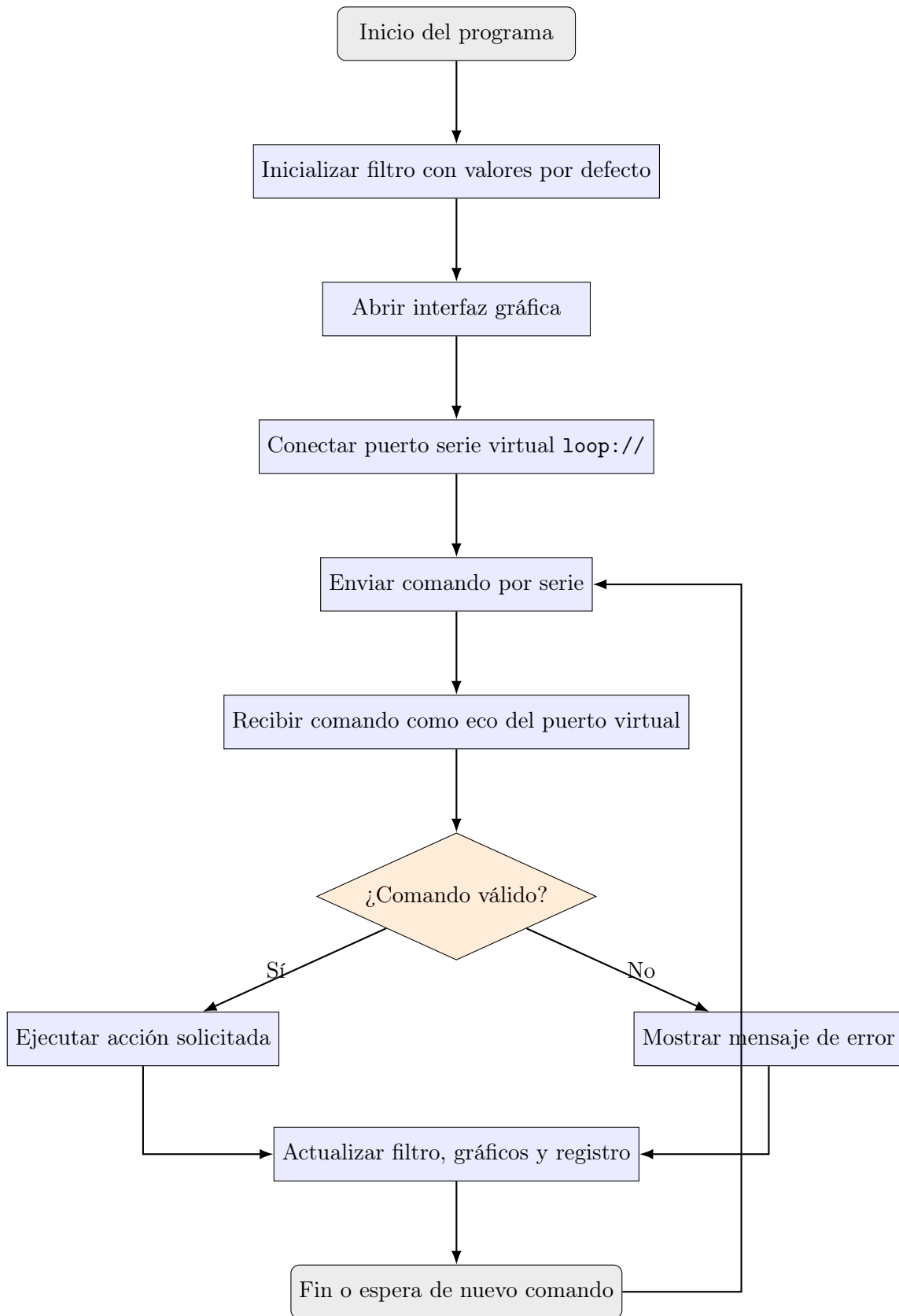


Figura 1: Diagrama de flujo general del programa.

7. Interfaz gráfica desarrollada

En la Figura 2 se muestra la interfaz gráfica implementada para el trabajo. En la parte superior se encuentran los controles de configuración del filtro y los botones de acción. En la zona central se observan los gráficos principales: respuesta temporal discreta, respuesta en frecuencia, comparación de filtros y señal BPSK filtrada. En la parte inferior se incluye el registro de comandos serie y respuestas del sistema.

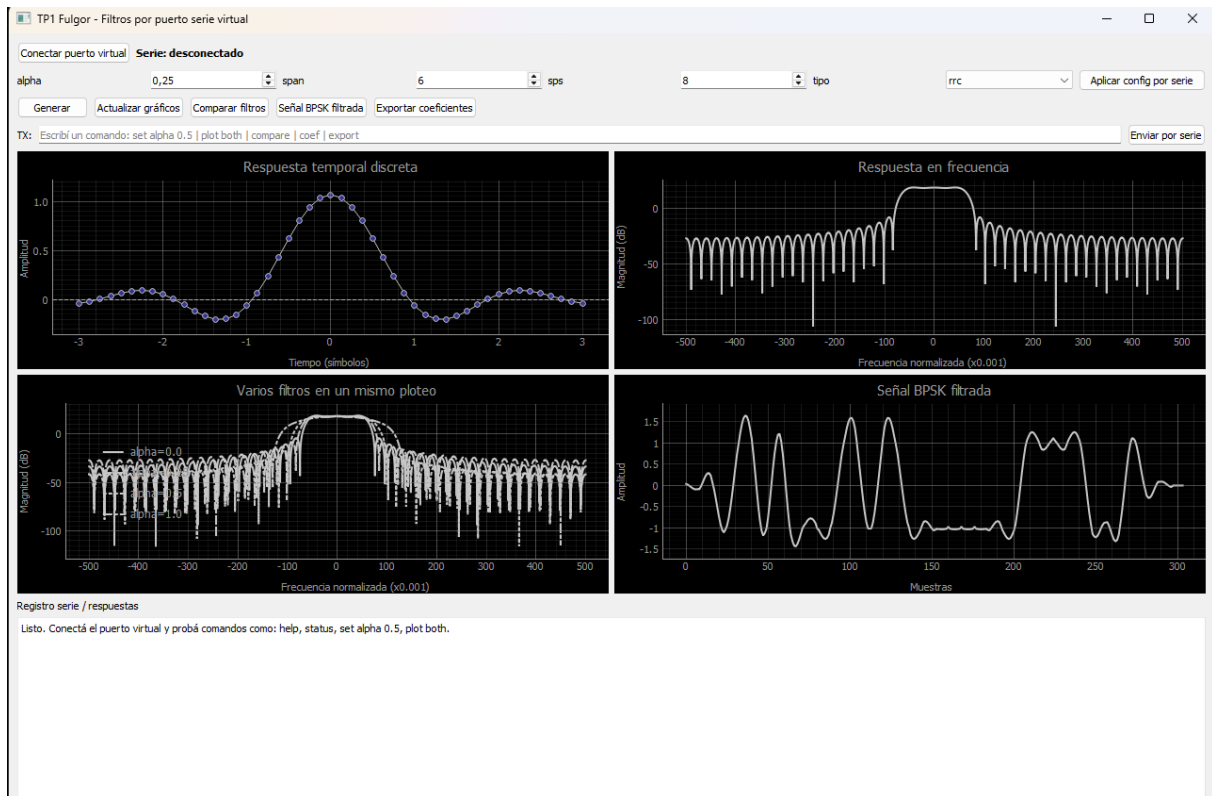


Figura 2: Interfaz gráfica del programa desarrollado.

8. Resultados obtenidos

Con los valores iniciales del programa se generó un filtro Root Raised Cosine con:

$$\alpha = 0.25, \text{ span} = 6, \text{ sps} = 8$$

La respuesta temporal se muestra de forma discreta, utilizando muestras individuales del filtro. La respuesta en frecuencia permite observar el comportamiento del filtro en el dominio frecuencial. Además, la comparación de filtros permite analizar cómo cambia la respuesta al variar el factor de roll-off α .

La señal BPSK filtrada permite visualizar una aplicación práctica del filtro, mostrando cómo los símbolos digitales son suavizados por la respuesta impulsional del filtro.

9. Instalación y ejecución

Para instalar las librerías necesarias se utiliza:

```
python -m pip install -r requirements.txt
```

Para ejecutar la interfaz gráfica:

```
python app_gui.py
```

Para ejecutar la versión por consola:

```
python main.py
```

10. Conclusión

El trabajo desarrollado cumple con los objetivos planteados, ya que permite generar filtros digitales, modificar sus parámetros mediante comunicación serie, visualizar distintas respuestas y comparar configuraciones. El uso del puerto serie virtual permite validar la lógica del sistema sin hardware físico, manteniendo igualmente la estructura de comunicación solicitada.

La interfaz gráfica facilita la interpretación de los resultados porque agrupa en una misma ventana la configuración del filtro, los comandos serie, las respuestas temporales y frecuenciales, la comparación entre filtros y una señal BPSK filtrada. De esta forma, el programa no sólo cumple la consigna técnica, sino que también presenta los resultados de manera clara y ordenada.